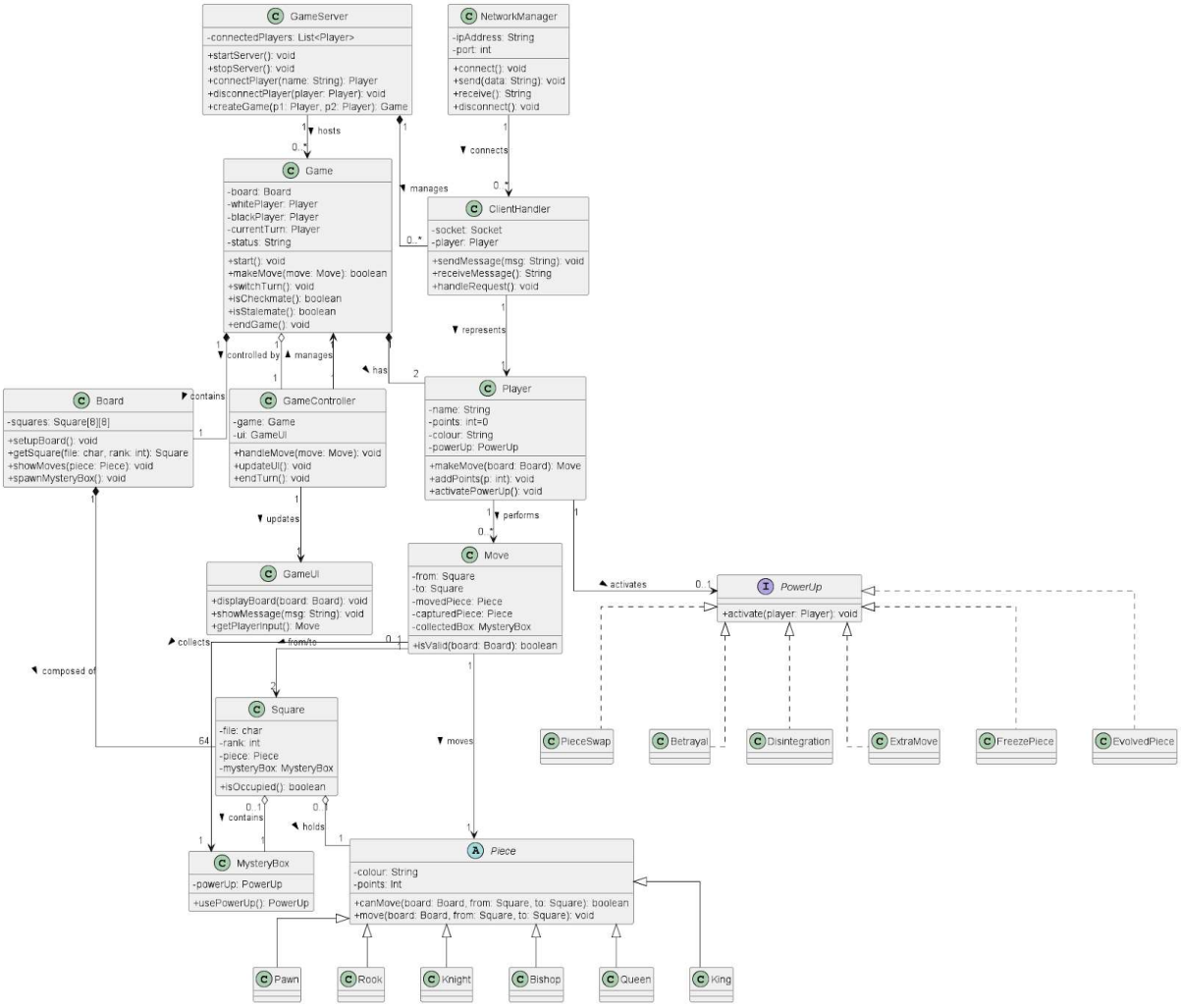
Contents Page

1. Refined Class Diagram ..... Page 2
2. Object Diagrams ..... Page 3 - 4
3. User Interface Mock-ups ..... Page 5 - 7

   3A. Network Ideas (Deployment Diagram) ..... Page 8 - 9
4. State Machines ..... Page 10 - 11
5. Sequence Diagrams ..... Page 12 - 14
6. Collaboration (Communication) Diagrams ..... Page 15 - 16
7. Class Skeletons ..... Page 17 - 31
8. Team Meeting Minutes ..... Page 31

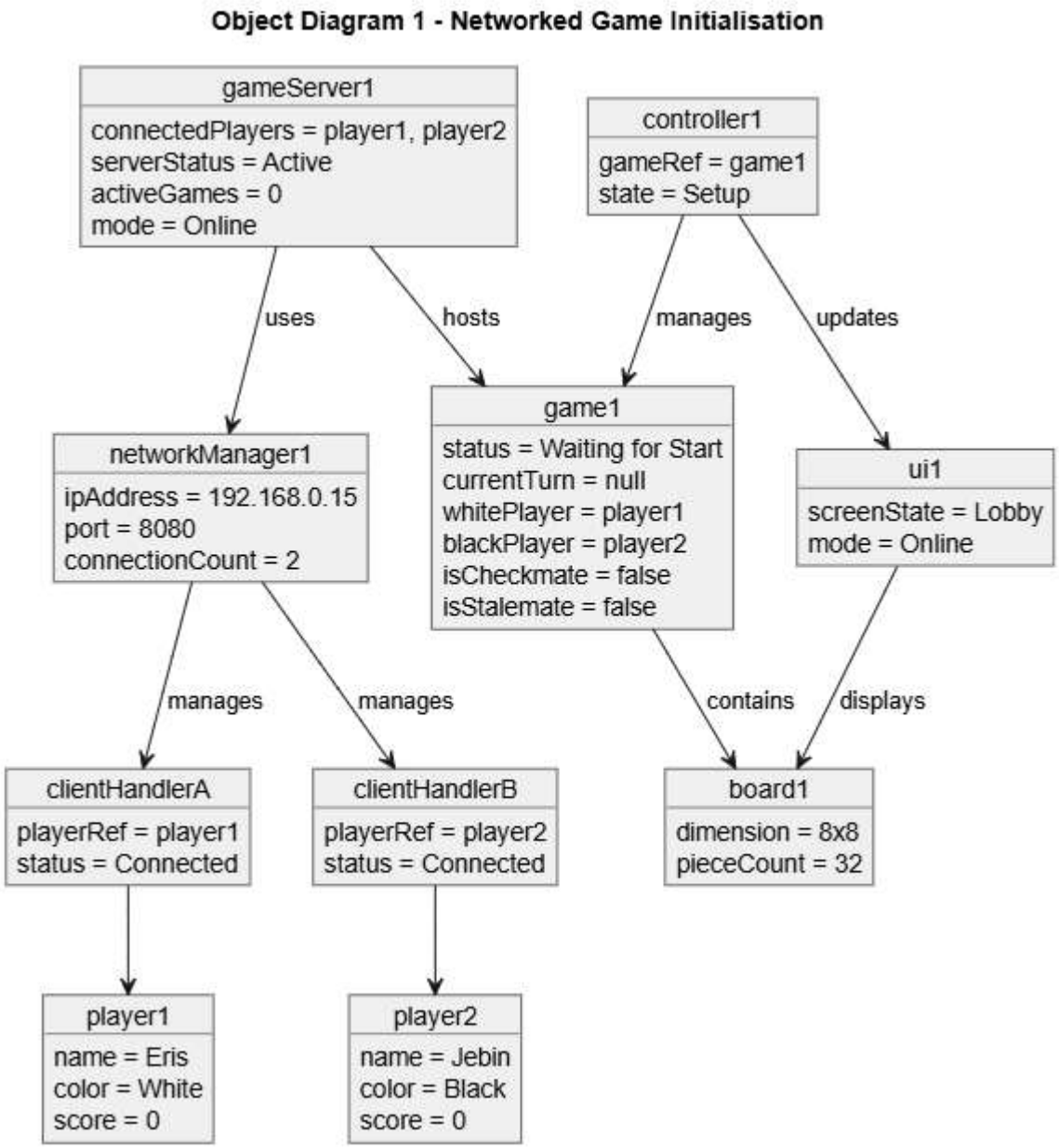
1. Refined Class Diagram



2. Object diagrams

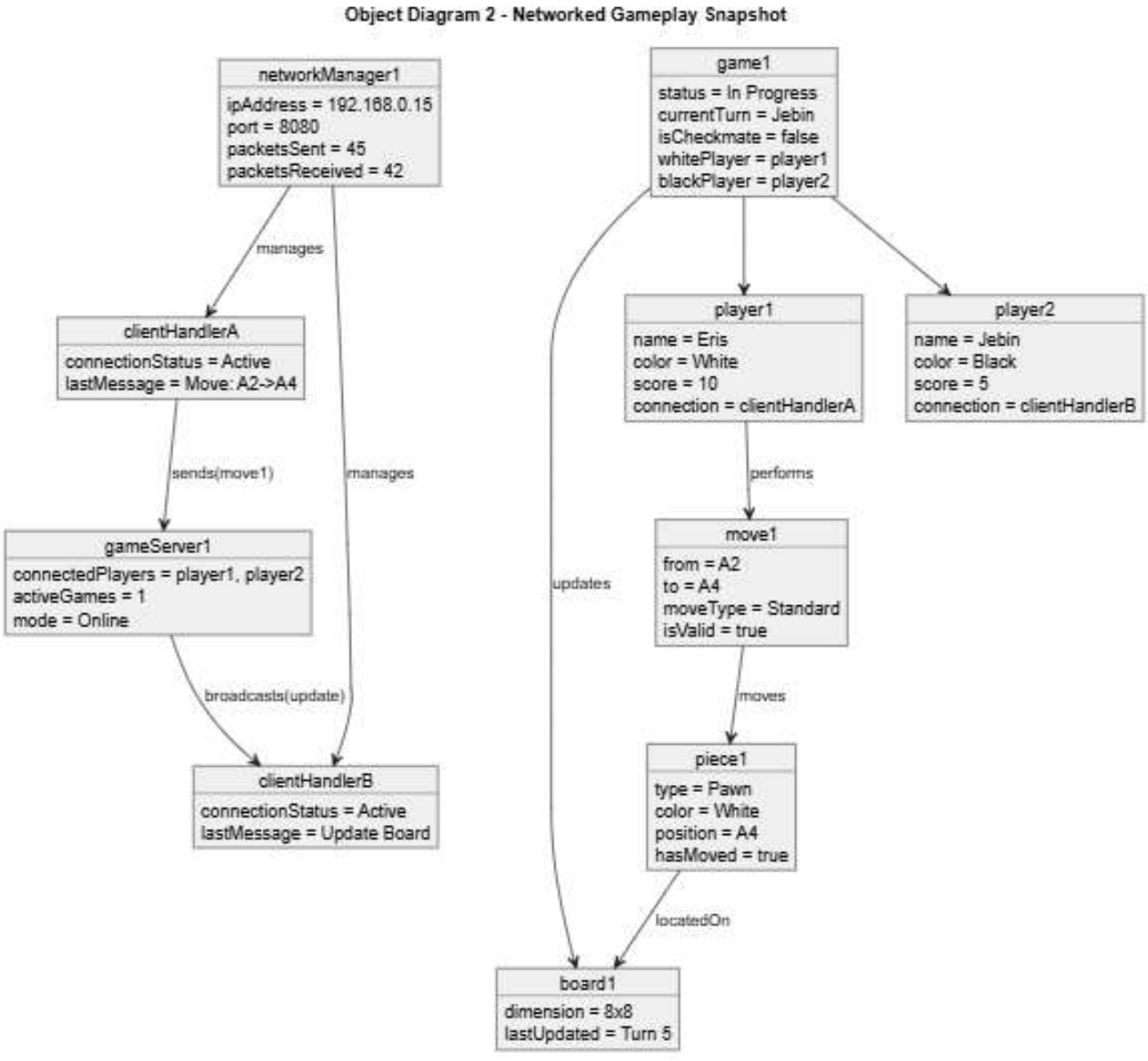
2.1 Networked Game Initialisation:

This diagram depicts the online setup using NetworkManger and ClientHandler to support online connectivity between two players, assigning both an IP address and a port to communicate with one another. It also highlights the steps it would take to start the game.



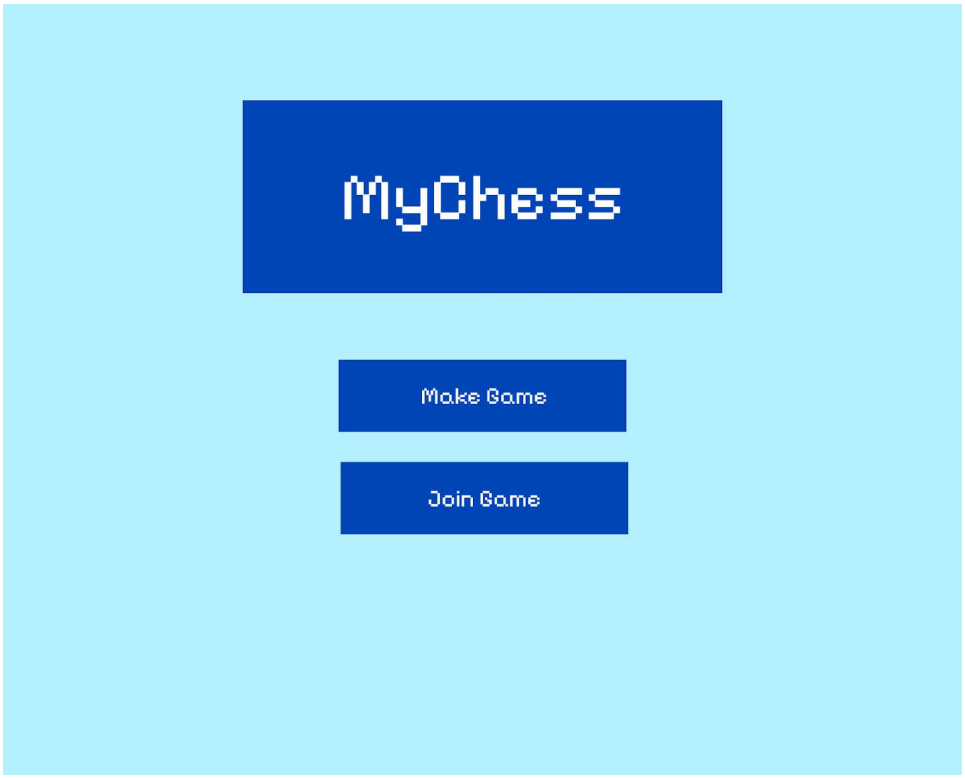
2.2 Networked Gameplay Snapshot:

Outlines the steps taken locally when two players compete against each other. It includes network attributes that demonstrate real-time gameplay synchronisation.

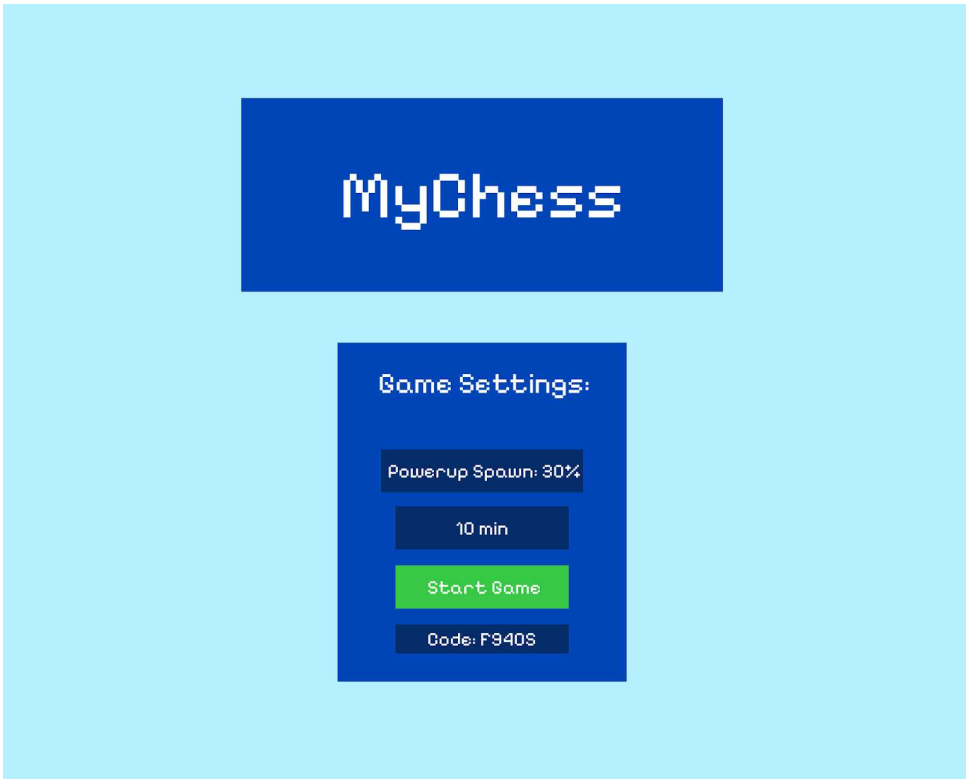


3. User Interface Mock-ups

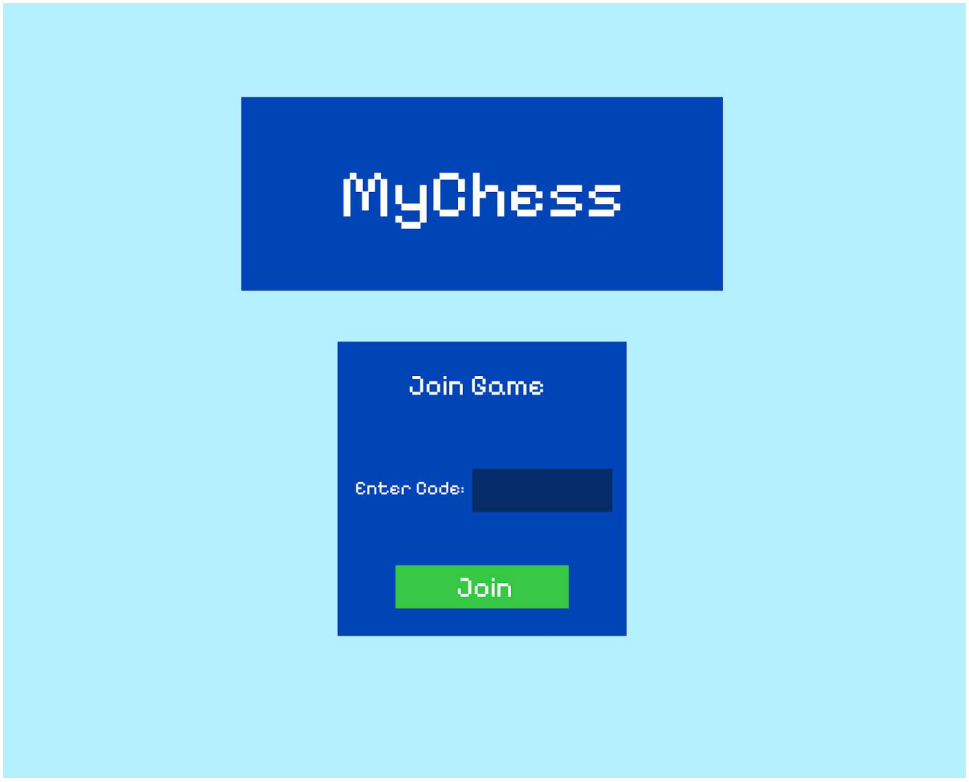
3.1 Menu UI Mockup



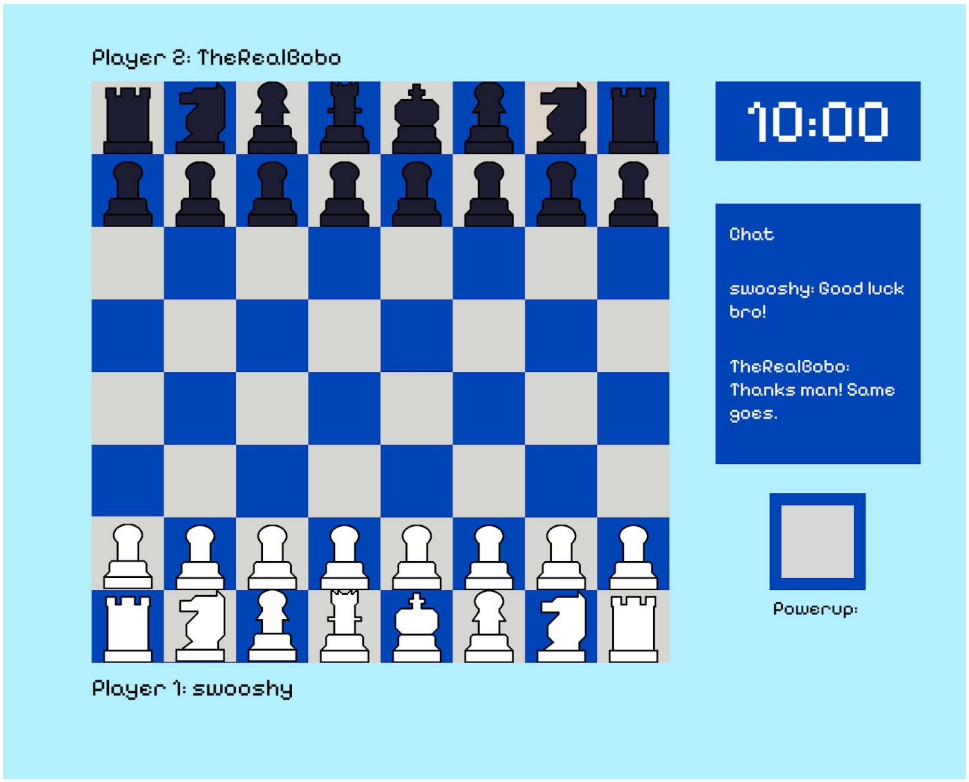
3.2 Create Game UI Mockup



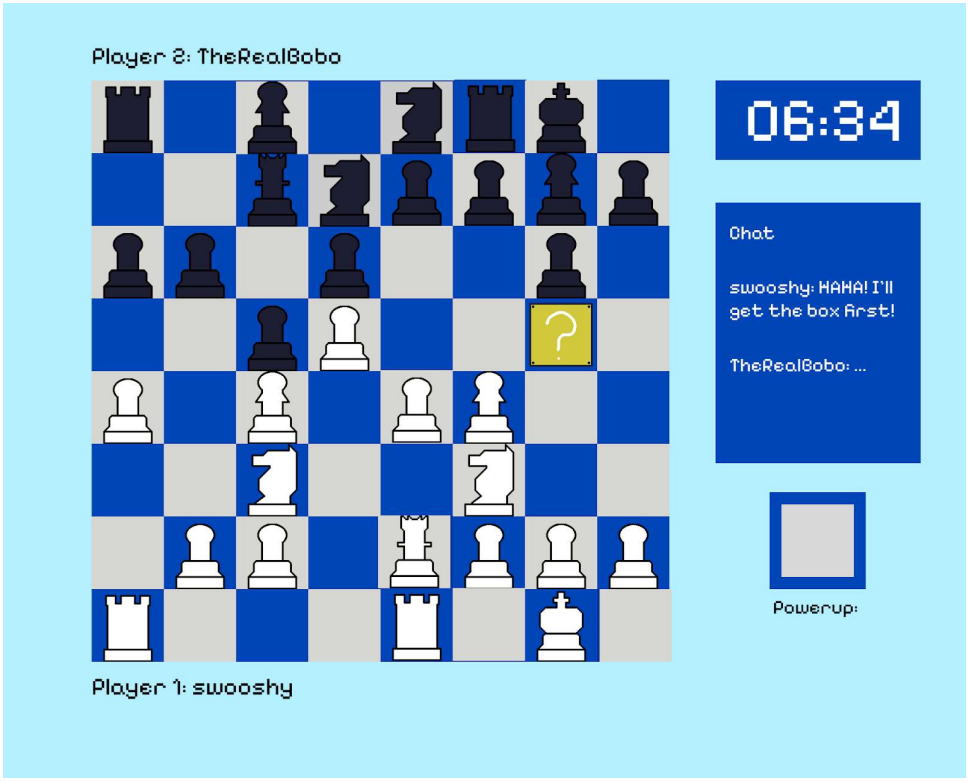
3.3 Join Game UI Mockup



3.4 Board UI Mockup



3.5 Game w/ Powerup UI Mockup



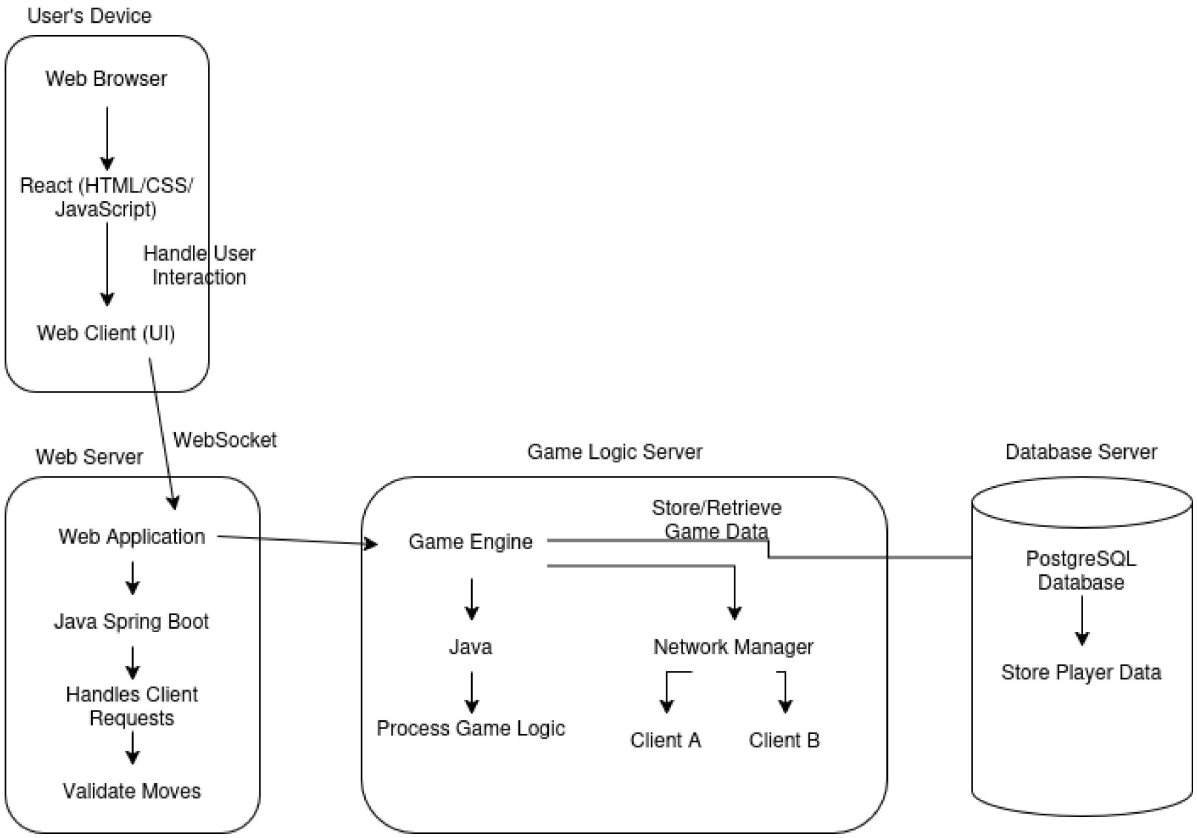
3.6 Game w/ Powerup Obtained UI Mockup



3A. Network Ideas (Deployment Diagram)

We plan to use websockets to handle real-time gameplay communication between clients and the server. We would use a game server to run and validate all the game logic throughout the game. We would need a database server to store user' data (name, profile picture, statistics).

Deployment Diagram



Server Side

```java
package com.example.gameserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
@RestController
public class GameServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(GameServerApplication.class, args);
    }

    @GetMapping("/welcome")
    public String welcome() {
        return "Welcome to the Game Server!";
    }
}
```

Client Side

```java
package com.example.gameclient;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class GameClient {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://localhost:12345/welcome");
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");

            BufferedReader in = new BufferedReader(new InputStreamReader(conn.getInputStream()));
            System.out.println(in.readLine());

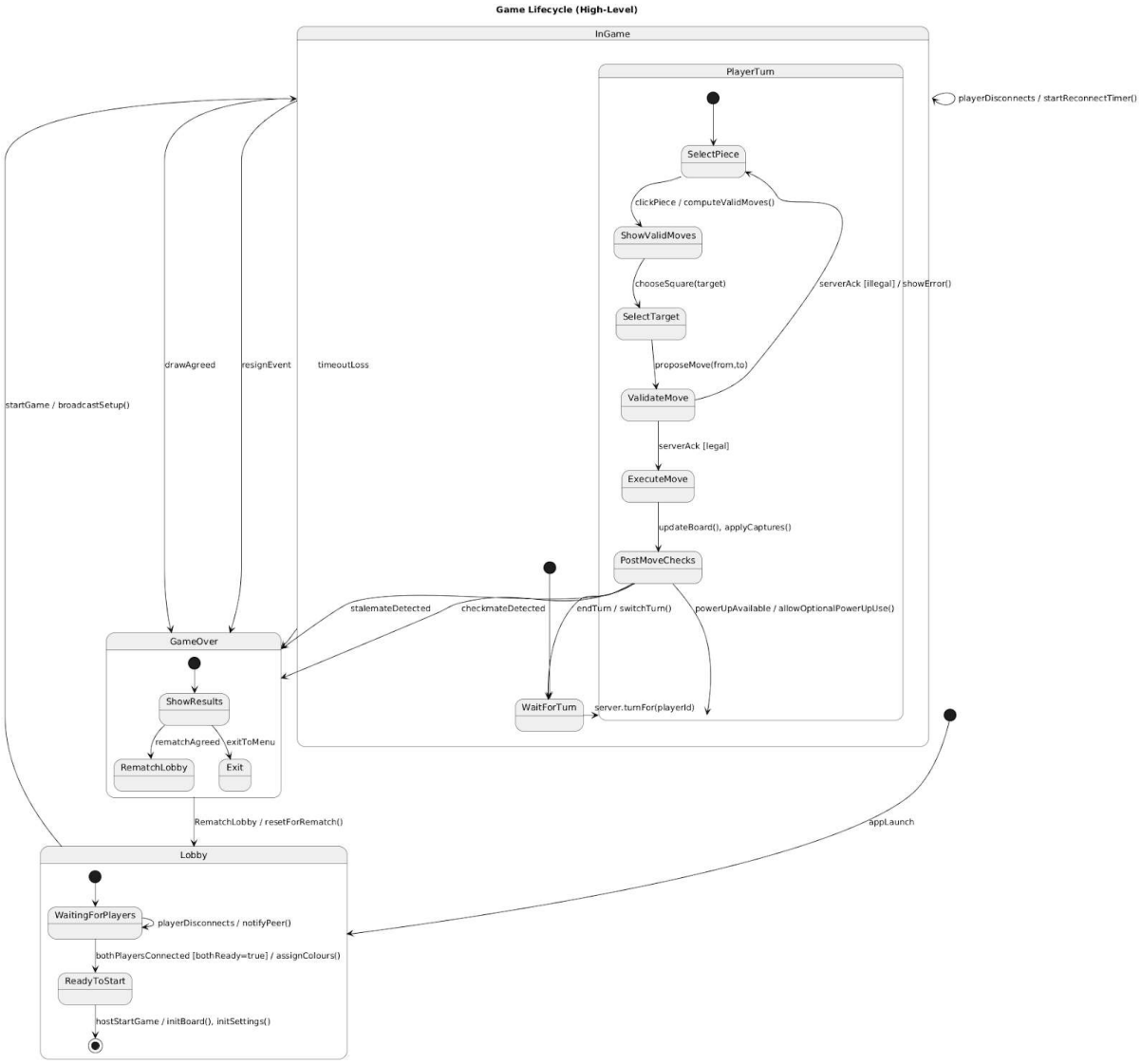
            in.close();
            conn.disconnect();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. State Machines

4.1. Game Lifecycle Main State Diagram

This state machine models the overall lifecycle of the game from start to finish. It begins in the Lobby, where players join, connect, and mark readiness. Once both are ready, the system transitions into the InGame state, where gameplay alternates between players through the “Wait for Turn” and “Player Turn” sub-states.

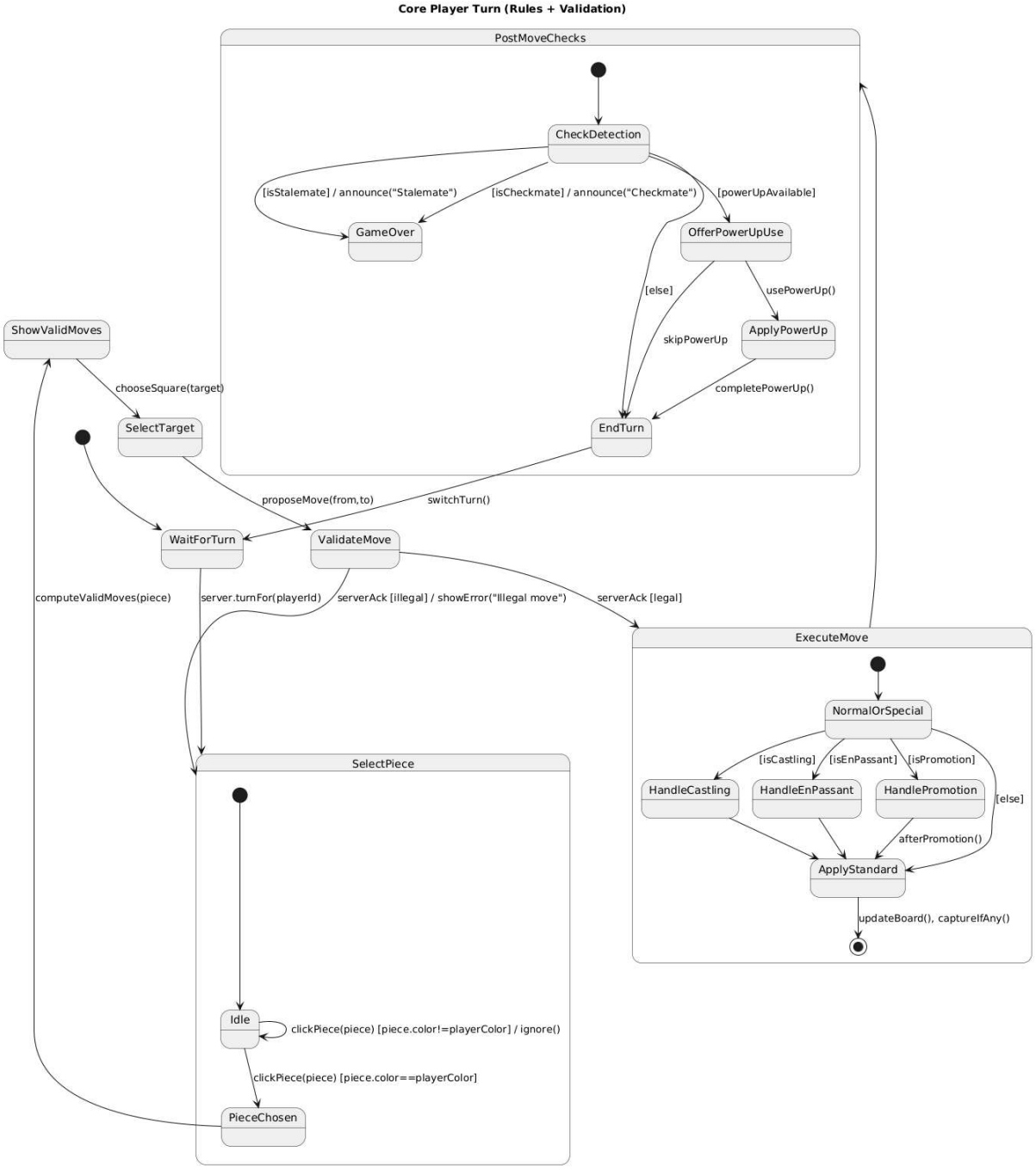
It includes transitions for resignations, disconnections, timeouts, draws, and reaching Game Over, where results are displayed and players may request a rematch.
Each transition is labeled with trigger events, guards (e.g., [bothReady=true]), and actions (e.g., / assignColours()).



4.2. Player Turn Sub-State Diagram

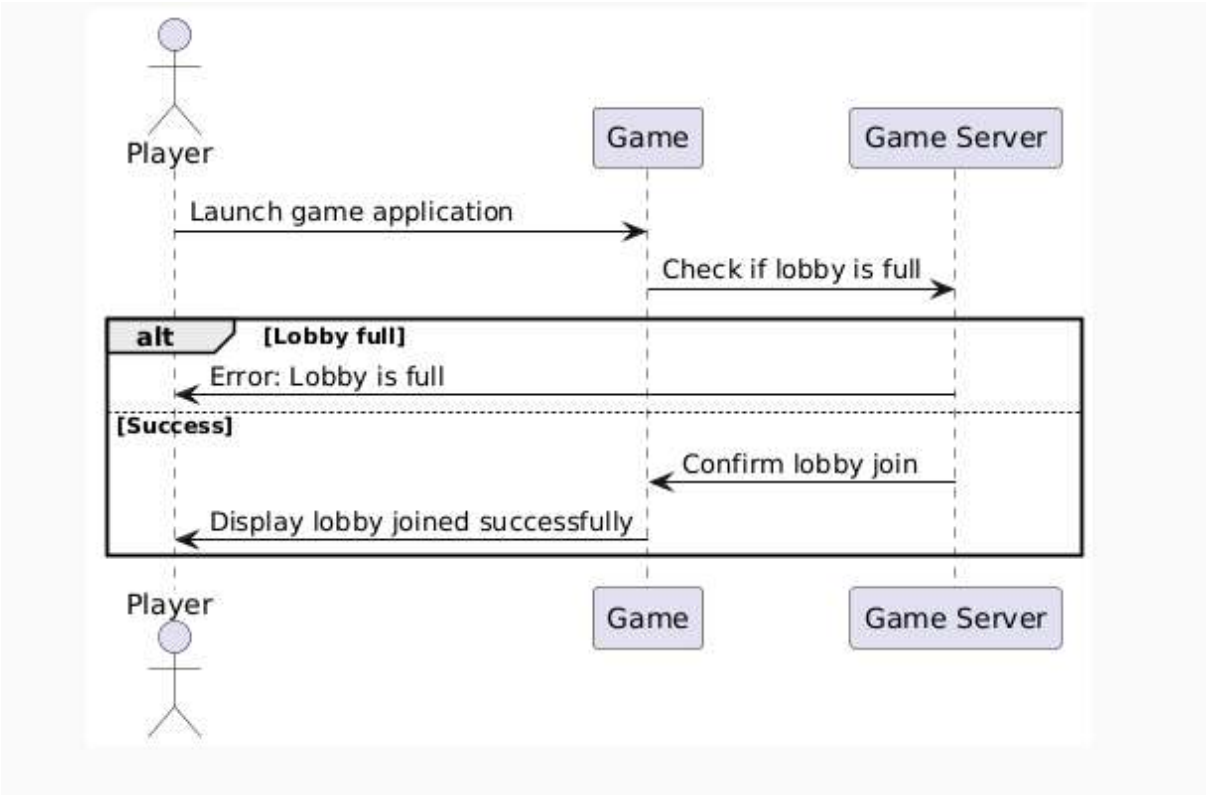
This sub-state machine zooms into the “Player Turn” phase from the main lifecycle. It represents the detailed decision logic when a player makes a move: selecting a piece, showing valid moves, choosing a target square, validating legality with the server, executing the move, and checking for checkmate or stalemate.

It also models special rules (castling, en passant, promotion) and optional variant actions such as power-ups. Each transition includes clear events, guards, and actions to demonstrate correctness and precision (e.g., serverAck [legal] / updateBoard()).

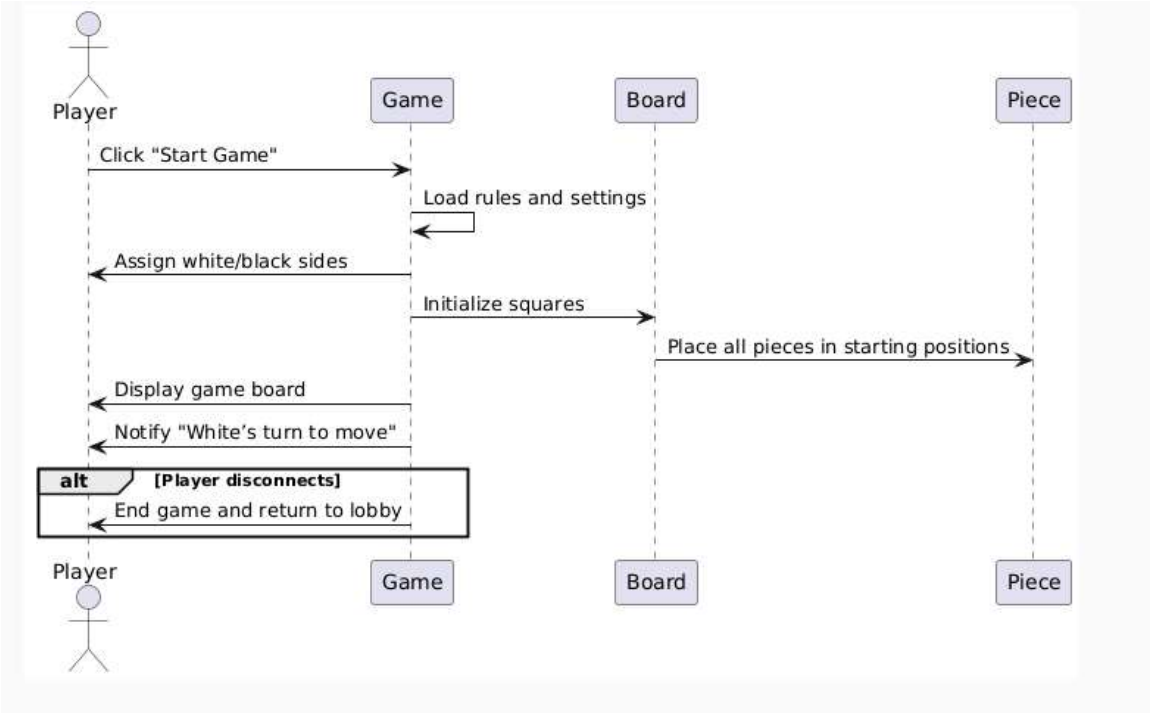


5. Sequence Diagrams

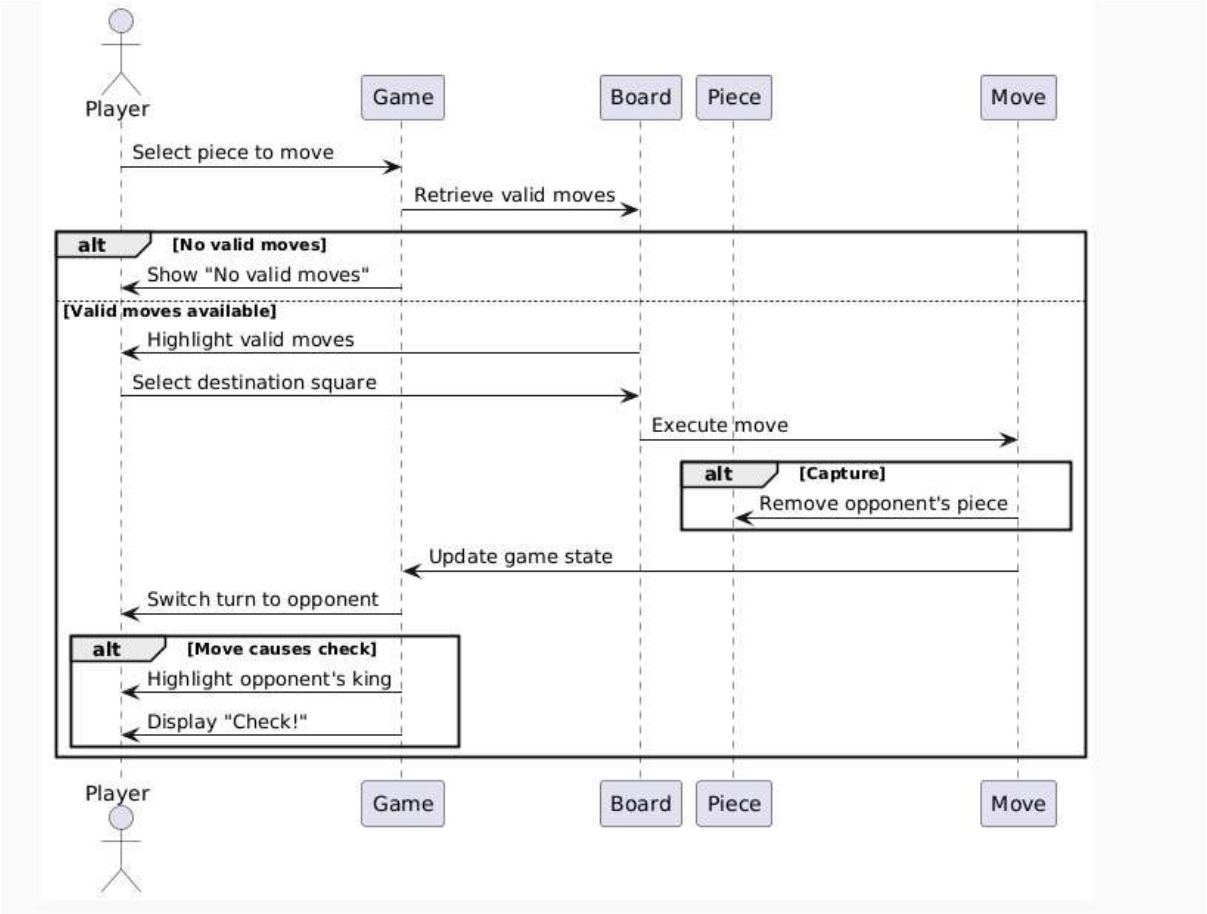
5.1. Player joins lobby



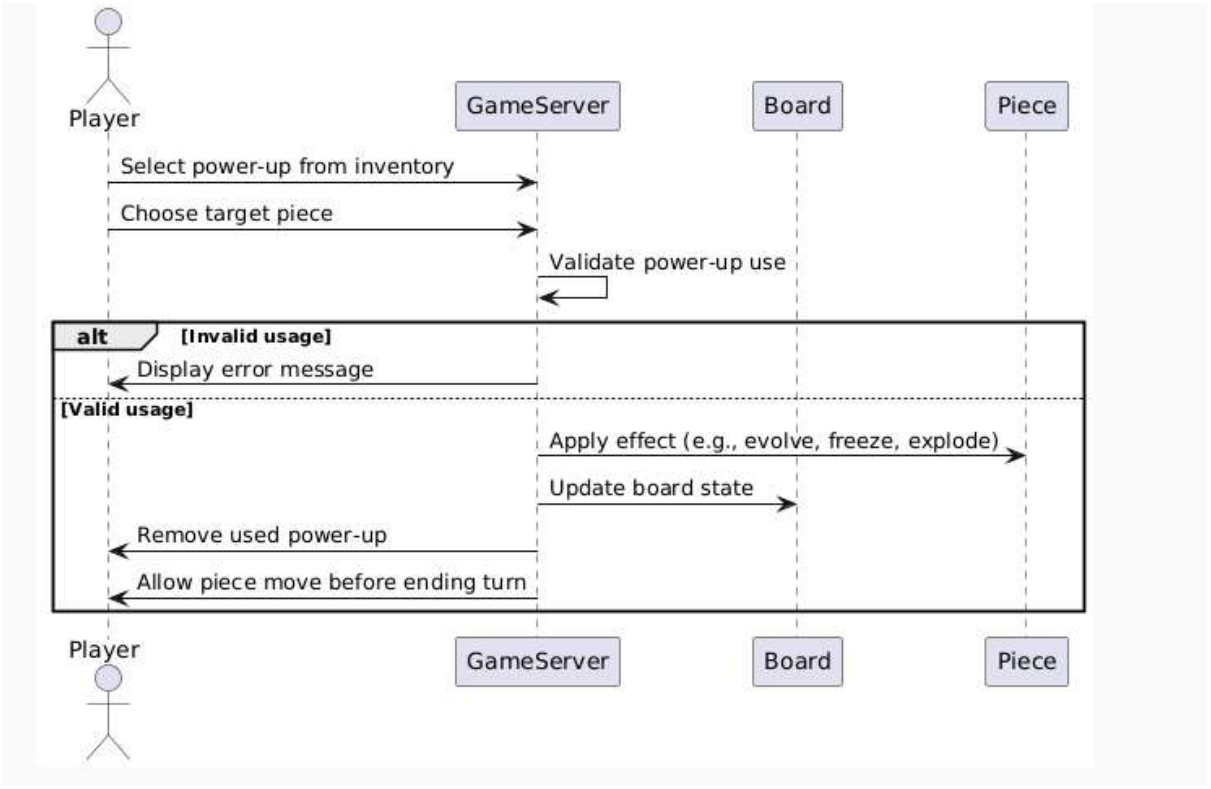
5.2. Game setup



5.3. Gameplay

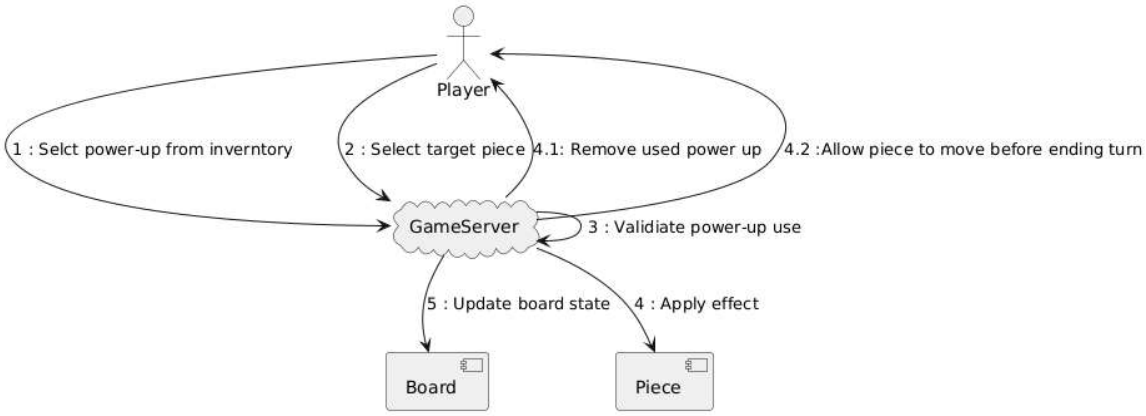


5.4 Using a Power-Up



6. Collaboration (Communication) Diagrams




Player
1 : Selct power-up from inverntory
2 : Select target piece
4.1: Remove used power up
4.2 :Allow piece to move before ending turn
GameServer
3 : Validiate power-up use
5 : Update board state
4 : Apply effect
Board
Piece

7. Class Skeletons

Class: GameServer

Roles:

- Handles all player connections and hosted games
- Transfer network messages between players

Object Creation & Deletion:

- Created when the server runs
- Deleted when the server shuts down

Semantics for Roles:

- Maintains list of connected players
- Only one instance of GameServer runs at a time
- Creates a new Game when a player creates a lobby

Attributes:

```
private List<Player> connectedPlayers;
// Stores all currently connected players
```

Constructor:

```
public GameServer() {
    // Pre-condition: Server environment is active
    // Post-condition: GameServer object is created with empty connectedPlayers list.
}
```

Methods:

```
public void startServer();
/* Semantics: Initialises socket and listens for incoming connections
 * Pre-condition: Port and IP address must be valid
 * Post-condition: Server listens for incoming connections
*/

public void stopServer();
/* Semantics: Stops server and disconnects all players
 * Pre-condition: Server must be running
 * Post-condition: Server is shutdown and all players are disconnected
 */

public Player connectPlayer(String name);
/* Semantics: Connects a player to the server
 * Pre-condition: Server is running and player is not already connected
 * Post-condition: Player is added to connectedPlayers list
 */
```

```
public void disconnectPlayer(Player player);
/* Semantics: Disconnects a player from the server
 * Pre-condition: Server is running and player is already connected
 * Post-condition: Player is removed from connectedPlayers list
 */

public Game createGame(Player p1, Player p2);
/* Semantics: Creates a new chess match between two players
 * Pre-condition: Both players are connected and not in another game
 * Post-condition: A new Game object is instantiated and managed by the server
 */
```

Class: NetworkManager

Roles:

- Handle connection between client and server
- Provide operations such as connect/send/receive/disconnect
- Make sure players' game states are synchronised

Object Creation & Deletion:

- Created when new session for Game is needed
- Deleted when session ends or application shuts down

Semantics for Roles:

- NetworkManager owns 0 or more ClientHandler objects which handle I/O
- Handles requests for one endpoint (ipAddress, port)

Attributes:

```
private String ipAddress;
// Destination IP Address

private int port;
// Destination port number
```

Constructor:

```
public NetworkManager(String ipAddress, int port) {
    /* Pre-condition - ipAddress is valid, port is within valid range
     * Post-condition - NetworkManager instance created
     */
}
```

Methods:

```java
public void connect();
/* Semantics - Opens the connection
 * Pre-condition - Not already connected
 * Post-condition - Socket established and ready for I/O
 */

public void send(String data);
/* Semantics - Sends message to endpoint
 * Pre-condition - Connection is open and data is not null
 * Post-condition - Message sent to server
 */

public String receive();
/* Semantics - Receive message from endpoint
 * Pre-condition - Connection is open
 * Post-condition - Returns message as a string
 */

public void disconnect();
/* Semantics - Closes the connection
 * Pre-condition - Connection is open
 * Post-condition - Socket closed. All ClientHandlers stop I/O
 */
```

Class: Game

Roles:

- Handles match between two players
- Handle game logic, validate and apply moves

Object Creation & Deletion:

- Created when two players are matched
- Deleted when match ends (checkmate, resignation, stalemate)

Semantics for Roles:

- Exactly two players needed for Game (one is white, one black)
- Only one player may move at a time

Attributes:

```
private Board board;
// The current game board holding all squares and pieces

private Player whitePlayer;
private Player blackPlayer;
// Two players playing, one white and one black

private Player currentTurn;
// Current player's turn

private String status;
// Game status (ONGOING, CHECKMATE, STALEMATE, RESIGNATION)
```

Constructor:

```
public Game(Player p1, Player p2) {
    /* Pre-condition - Players p1 and p2 are valid
     * Post-condition - Game object created (game not started yet)
     */
}
```

Methods:

```
public void start();
/* Semantics - Start a new game
 * Pre-condition - Game status is NOTSTARTED
 * Post-condition - New Board is initialised, players are assigned
 *                  a colour each, white starts and status is set to ONGOING
 */

public boolean makeMove(Move move);
/* Semantics - Validates and applies a move
 * Pre-condition - It is currentTurn's move and move is legal for the piece
 * Post-condition - Board updated and turn changes to the other player
 */

public void switchTurn();
/* Semantics - Switches turn to the opponent
 * Pre-condition - Last move successfully applied
 * Post-condition - currentTurn switched
 */
```

```
public boolean isCheckmate();
/* Semantics - Checks if checkmate is on the board
 * Pre-condition - Game is ongoing and board state is valid
 * Post-condition - Returns whether or not it is checkmate
 */

public boolean isStalemate();
/* Semantics - Checks if stalemate is on the board
 * Pre-condition - Game is ongoing and board state is valid
 * Post-condition - Returns whether or not it is stalemate
 */

public void endGame();
/* Semantics - Ends match. No further moves can be made
 * Pre-condition - Game is ongoing
 * Post-condition - Game ends, results displayed
 */
```

Class: ClientHandler

Roles:

- Manages a single client connection
- Handle send/receive operations for one player
- Listen for I/O

Object Creation & Deletion:

- Created when a client connects to server
- Deleted when client disconnects or server shuts down

Semantics for Roles:

- One ClientHandler corresponds to exactly one Player
- Responsible for message transfer between client and server

Attributes:

```
private Socket socket;
// Network socket used to communicate between server and client

private Player player;
// Player that this ClientHandler corresponds to
```

Constructor:

```
public ClientHandler(Socket socket, Player player) {
    /* Pre-condition - socket and player are valid
     * Post-condition - ClientHandler object listens to player I/O
     */
}
```

Methods:

```
public void sendMessage(String msg);
/* Semantics - Sends message to connected client
 * Pre-condition - Connection active + msg not null
 * Post-condition - Message transmitted to client
 */

public String receiveMessage();
/* Semantics - Waits for and returns next message from the client
 * Pre-condition - Connection active
 * Post-condition - Returns received message string
 */

public void handleRequest();
/* Semantics - Interprets received request
 * Pre-condition - The request cannot be null
 * Post-condition - Appropriate server action is triggered
 */
```

Class: Player

Roles:

- Represents a player in a chess game
- Store details about the player corresponding to the game
- Perform moves and activate power-ups

Object Creation & Deletion:

- Created when a player joins a game
- Deleted when a player leaves a game or the game ends

Semantics for Roles:

- A player may collect and hold up to one PowerUp at a time
- Points are accumulated through taking opponent's pieces

Attributes:

```
private String name;
// The player's name

private int points = 0;
// Amount of points the player has

private String colour;
// Colour (white or black)

private PowerUp powerUp;
// Refers to the player's currently held power-up
```

Constructor:

```
public Player(String name, String colour) {
    /* Pre-condition - name and colour are valid and non-null
     * Post-condition - Player object created with points = 0 and no powerUp
     */
}
```

Methods:

```
public Move makeMove(Board board);
/* Semantics - Makes a move on the board
 * Pre-condition - Must be this player's turn
 * Post-condition - Move object representing information about the move is returned. Board updated
 */

public void addPoints(int p);
/* Semantics - Adds p amount of points to the player's score
 * Pre-condition - p must be positive integer
 * Post-condition - points increases by p
 */

public void activatePowerUp();
/* Semantics - Activates the player's currently stored PowerUp
 * Pre-condition - Valid PowerUp exists
 * Post-condition - PowerUp effect executed and PowerUp removed from player's inventory
 */
```

Class: Board

Roles:

- Represent chessboard as 8x8 grid of Square objects
- Maintain positions of all pieces throughout the game
- Display valid moves and handle spawning mystery boxes

Object Creation & Deletion:

- Created when a new game starts

- Deleted when game ends

Semantics for Roles:

- Board consists of 64 Square objects in an 8x8 grid
- Only one active Board per Game

Attributes:

```
private Square[][] squares = new Square[8][8];
// Stores 8x8 array of squares. Each square can hold a Piece or be empty
```

Constructor:

```
public Board() {
    /* Pre-condition - None
     * Post-condition - New empty board is created
     */
}
```

Methods:

```
public void setupBoard();
/* Semantics - Set up white and black pieces in starting squares
 * Pre-conditions - squares array must be initialised
 * Post-conditions - Board is set up in starting state
 */

public Square getSquare(char file, int rank);
/* Semantics - Returns the Square at the specified file and rank
 * Pre-conditions - file and rank must be in valid range (a-h, 1-8)
 * Post-conditions - Returns valid Square object
 */

public void showMoves(Piece piece);
/* Semantics - Show valid moves for a given piece
 * Pre-conditions - Piece is valid and exists on the board
 * Post-conditions - Possible squares the piece can move to are displayed
 */

public void spawnMysteryBox();
/* Semantics - Spawn a MysteryBox in a random location on the board
 * Pre-conditions - There must be atleast one empty square available and
 *                  there must only be one MysteryBox at a time
 * Post-conditions - A MysteryBox is placed and can be collected by a player
 */
```

Class: GameController

Roles:

- Intermediary between Game and GameUI
- Process player inputs and update UI

Object Creation & Deletion:

- Created when a Game is initialised
- Deleted when game ends or controller is reset

Semantics for Roles:

- Each Game has exactly one GameController
- Controller calls GameUI methods to update display after each turn or event

Attributes:

```
private Game game;
// The Game this GameController corresponds to

private GameUI ui;
// The GameUI this GameController corresponds to
```

Constructor:

```
public GameController(Game game, GameUI ui) {
    /* Pre-condition - game and ui must be valid
     * Post-condition - New GameController instance is created, linking game logic and UI
     */
}
```

Methods:

```
public void handleMove(Move move);
/* Semantics - Processes move, validates it and updates game
 * Pre-condition - Move is valid and game is ongoing
 * Post-condition - Board state and current turn updated and UI is refreshed
 */

public void updateUI();
/* Semantics - Refreshes UI to reflect latest game state
 * Pre-condition - ui is valid
 * Post-condition - UI shows most recent game state
 */

public void endTurn();
/* Semantics - Ends the current player's turn and moves to the opponent
 * Pre-condition - Valid move made
 * Post-condition - Game advances to next player's turn
 */
```

Class: GameUI

Roles:

- Present current game state to player
- Collect player input

Object Creation & Deletion:

- Created when a new match begins
- Deleted when game ends or UI is closed

Semantics for Roles:

- Must render the board after every game state change

Attributes:

(no attributes as shown in class diagram)

Constructor:

```
public GameUI() {
    /* Pre-condition - A valid game is ongoing
     * Post-condition - UI instance ready to display game + capture user input
     */
}
```

Methods:

```
public void displayBoard(Board board);
/* Semantics - Displays the given board
 * Pre-condition - Board is valid
 * Post-condition - UI updated to show the current game state
 */

public void showMessage(String msg);
/* Semantics - Displays a given message to the screen
 * Pre-condition - msg is non-null
 * Post-condition - Message appears on the UI
 */

public Move getPlayerInput();
/* Semantics - Records player's intended move
 * Pre-condition - It is the player's turn
 * Post-condition - Returns Move object representing input
 */
```

Class: Move

Roles:

- Represents a move made during a chess game
- Provide validation to confirm that move is valid

Object Creation & Deletion:

- Created when player selects a piece and target square
- Deleted when move has been applied to the board

Semantics for Roles:

- Each move must refer to exactly one source and one destination square

Attributes:

```
private Square from;
private Square to;
// Source and destination squares

private Piece movedPiece;
private Piece capturedPiece;
// Moved piece and captured piece (if any)

private MysteryBox collectedBox;
// Collected mystery box (if any)
```

Constructor:

```
public Move(Square from, Square to, Piece movedPiece) {
    /* Pre-condition - from, to, movedPiece must be non-null and valid
     * Post-condition - Move object created representing player's action
     */
}
```

Methods:

```
public boolean isValid(Board board);
/* Semantics - Determines whether the move is valid
 * Pre-condition - board is valid and contains the from and to squares
 * Post-condition - Returns true if move is valid and false otherwise
 */
```

Class: Square

Roles:

- Represents a square on the 8x8 chess board
- Store file, row, and any Piece or MysteryBox occupying it

Object Creation & Deletion:

- Created when a board is initialised
- Deleted when game ends

Semantics for Roles:

- Each square can contain a Piece, a MysteryBox or be empty.

Attributes:

```
private char file;
private int rank;
// The file and rank (column and row) of this square

private Piece piece;
// Chess piece currently occupying the square (if any)

private MysteryBox mysteryBox;
// MysteryBox currently occupying the square (if any)
```

Constructor:

```
public Square(char file, int rank) {
    /* Pre-condition - file must be between a-h and rank must be between 1-8
     * Post-condition - Square object is created with empty piece and mysteryBox
     */
}
```

Methods:

```
public boolean isOccupied();
/* Semantics - Checks if the current square is occupied
 * Pre-condition - Square is valid and is part of a board
 * Post-condition - Returns true if piece is valid otherwise false
 */
```

Class: MysteryBox

Roles:

- Represent a box that grants the player a PowerUp when picked up by a piece

Object Creation & Deletion:

- Created when Board spawns new mystery box (spawnMysteryBox())

- Deleted when a player collects it and PowerUp is given to the player

Semantics for Roles:

- Each MysteryBox contains exactly one PowerUp
- When collected, its PowerUp is transferred to the player and the box is removed from the board.

Attributes:

```
private PowerUp powerUp;
// The PowerUp stored in this mystery box
```

Constructor:

```
public MysteryBox(PowerUp powerUp) {
    /* Pre-condition - powerUp is valid
     * Post-condition - New MysteryBox created containing the specified PowerUp
     */
}
```

Methods:

```
public PowerUp usePowerUp();
/* Semantics - Returns the PowerUp stored in the box
 * Pre-condition - powerUp is not null
 * Post-condition - Returns powerUp and box is now empty
 */
```

Abstract Class: Piece

Roles:

- Abstract base class for every chess piece
- Defines shared attributes such as colour and points
- Provides core methods for moving a piece

Object Creation & Deletion:

- Created when Board is set up
- Deleted when captured, removed from play or game ends

Semantics for Roles:

- Every piece has a fixed colour (white or black)
- Subclasses must override canMove() to define unique movement rules

Attributes:

```
protected String colour;
// The colour of the piece (white or black)

protected int points;
// How many points the piece is worth
```

Constructor:

```
public Piece(String colour, int points) {
    /* Pre-condition - colour is "White" or "Black" and points is a positive integer
     * Post-condition - Initialises a Piece with specified colour and points
     */
}
```

Methods:

```
public abstract boolean canMove(Board board, Square from, Square to);
/* Semantics - Determines whether the piece can move from one square to another
 * Pre-condition - board, from, to must be valid and piece exists at from
 * Post-condition - Returns true if move is valid otherwise false
 */

public void move(Board board, Square from, Square to);
/* Semantics - Performs a given move if legal
 * Pre-condition - canMove(board, from, to) must return true
 * Post-condition - Piece is placed on destination square, source square becomes empty
 *                  and captured piece (if any) is removed
 */
```

Subclasses of Piece:

The classes Pawn, Rook, Knight, Bishop, Queen and King inherit from the abstract class Piece. Each overrides the canMove() method to implement its own unique movement rules.

Interface: PowerUp

Roles:

- Define a common interface for all power-ups used in the game

Semantics for Roles:

- Implemented by all power-up classes (PieceSwap, Betrayal, Disintegration, ExtraMove, FreezePiece, EvolvedPiece)

Methods:

```
public void activate(Player player);
/*
 * Semantics - Applies the specified power-up to the player
 * Pre-condition - player must be valid, in a game and holds the power-up
 * Post-condition - PowerUp's effect is applied
 */
```

8. Team Meeting Minutes

"1st Meeting" Started at 13:02 21/10/25. Ended at 13:31.

Attended by all members.

We revised the document of what our assignment specification is and general conversation of how to approach it. Began assigning sections based on previous assignment and members affinity to complete.

“2nd Meeting” Started at: 13:05 28/10/25. Ended at 14:02.

All members attended except but later informed online.

Updated each other on current work split. Ensuring earlier parts that later parts depend on are completed so that things come together seamlessly. Reassigned some members to additionally help with object diagram and class skeletons section. All members attended except who was later informed online.

“3rd Meeting” Started at: 12:06 30/10/25. Ended 13:33.

Attended by all members.

Stand up on all work accomplished currently. Debrief on issues brought up while doing assignment, discussed solutions for incompleted parts, otherwise mostly polishing assignment into a cohesive and comprehensive manner.